



POF INTERNSHIP REPORT

SUBMITTED BY:

AYESHA BAKHTAWAR

SUBMITTED TO:

SIR....

DISCIPLINE: BS(SOFTWARE ENGINEERING)

DURATION: 27 JULY 2026 – 06 SEPTEMBER 2026

HITEC UNIVERSITY, TAXILA

Day 1 – PHP & Laravel Fundamentals

1. Introduction to Web Development

Definition

Web development is the process of creating websites and web applications that run on the internet.

Types of Web Development

Frontend

Frontend is the visible part of a website that users can see and interact with.

Languages Used:

- HTML
- CSS
- JavaScript

Examples:

- Buttons
- Images
- Forms
- Navigation Menu

Backend

Backend is the hidden part of a website that works behind the scenes.

Languages Used:

- PHP
- Python
- Java
- C#

Backend Handles:

- Login System
- Database
- Payments
- Authentication
- Security

Database

A database stores all the website's information.

Examples:

- Students
- Teachers
- Products
- Orders

Laravel mostly uses **MySQL** as its database.

2. Client-Server Architecture

Definition

The Client-Server Architecture is a communication model where the client sends a request and the server processes it and returns a response.

Client

A client is the device or browser used by the user.

Examples:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

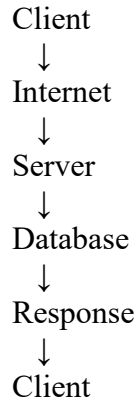
Server

A server is a computer that stores website files and data.

Examples:

- Apache
- Nginx

Flow



Example

When you open www.google.com, your browser sends a request to Google's server. The server processes the request and sends the webpage back to your browser.

3. HTTP

Full Form

Hyper Text Transfer Protocol

Definition

HTTP is a protocol used for communication between a web browser and a web server.

Features

- Transfers web pages.
- Not encrypted.
- Less secure.

Example

<http://example.com>

4. HTTPS

Full Form

Hyper Text Transfer Protocol Secure

Definition

HTTPS is the secure version of HTTP.

It encrypts data using an **SSL (Secure Sockets Layer) Certificate**.

Features

- Secure communication
- Encrypts data
- Protects passwords and personal information

Example

`https://google.com`

A lock icon appears in the browser.

Difference Between HTTP and HTTPS

HTTP	HTTPS
Not Secure	Secure
No Encryption	Uses SSL Encryption
Uses Port 80	Uses Port 443
No Lock Icon	Lock Icon
Less Safe	More Safe

5. MVC Architecture

Definition

MVC stands for **Model-View-Controller**.

Laravel follows the MVC architecture to organize code.

Model (M)

The Model communicates with the database.

Examples

- Student
- Teacher
- Course

Work

- Insert data
- Update data
- Delete data
- Fetch data

View (V)

The View is the user interface that users can see.

Laravel uses **Blade Templates**.

Example:

home.blade.php

The View displays data to the user.

Controller (C)

The Controller acts as the middle layer between the Model and the View.

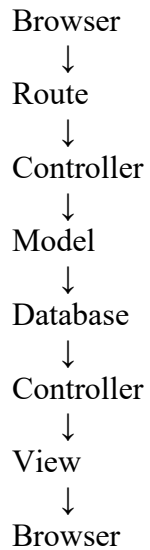
Functions

- Receives requests
- Processes data

- Calls the Model
- Returns the View

Example:

StudentController

MVC Flow**6. XAMPP****Definition**

XAMPP is a free software package that allows developers to run PHP and MySQL applications on their local computer.

Meaning of XAMPP

X = Cross Platform

Works on Windows, Linux, and macOS.

A = Apache

Runs the website.

M = MySQL

Stores website data.

P = PHP

Programming language used by Laravel.

P = Perl

Another programming language included with XAMPP.

Uses of XAMPP

- Run Laravel locally.
- Run PHP projects.
- Connect with MySQL database.
- Test websites before uploading them online.

7. Composer

Definition

Composer is a dependency manager for PHP.

It installs Laravel and manages PHP packages automatically.

Example Command

```
composer create-project laravel/laravel myproject
```

Uses

- Install Laravel
- Install packages
- Update packages
- Manage project dependencies

8. Visual Studio Code (VS Code)

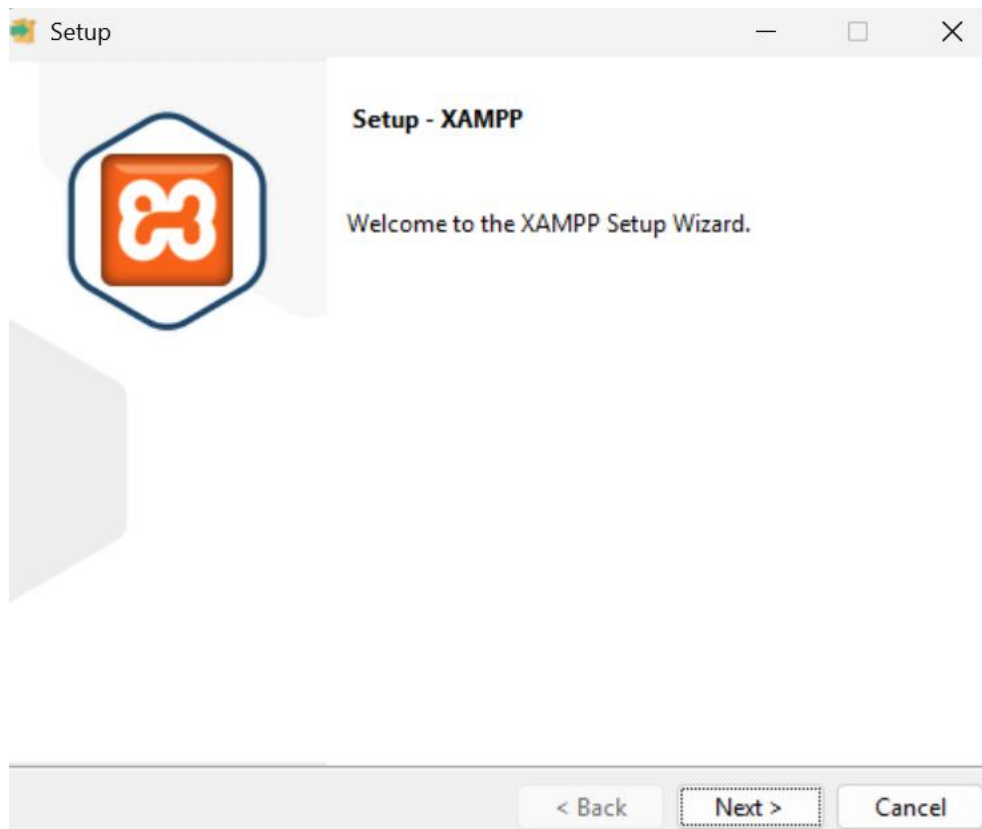
Definition

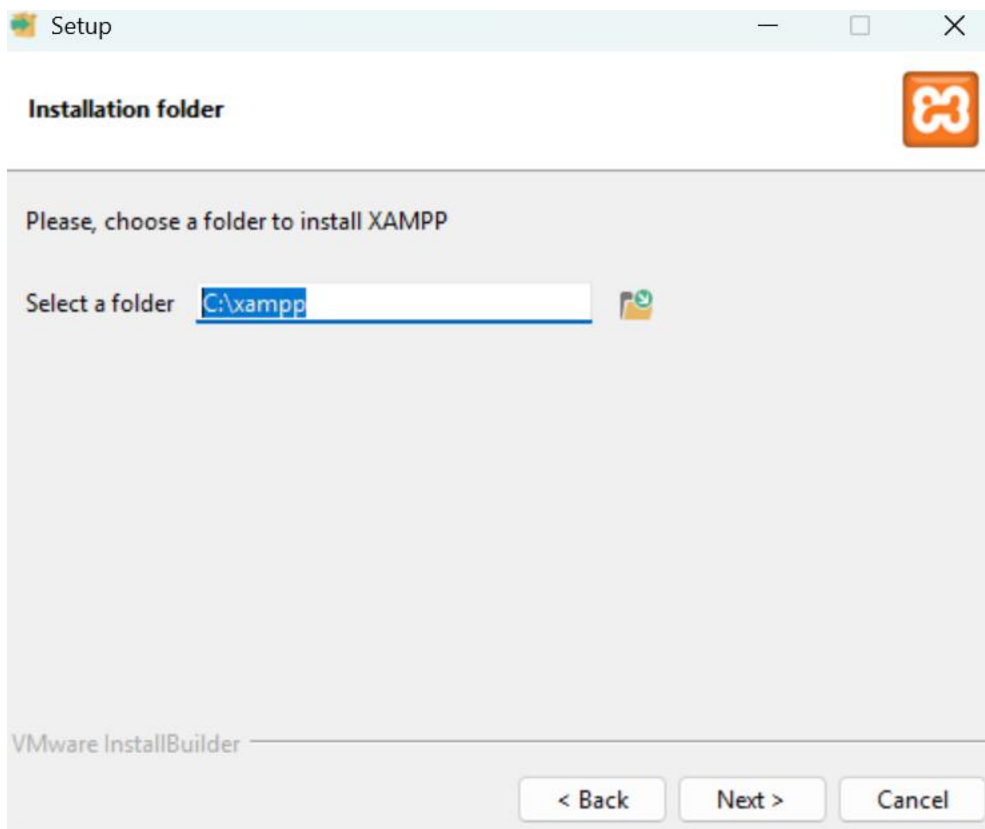
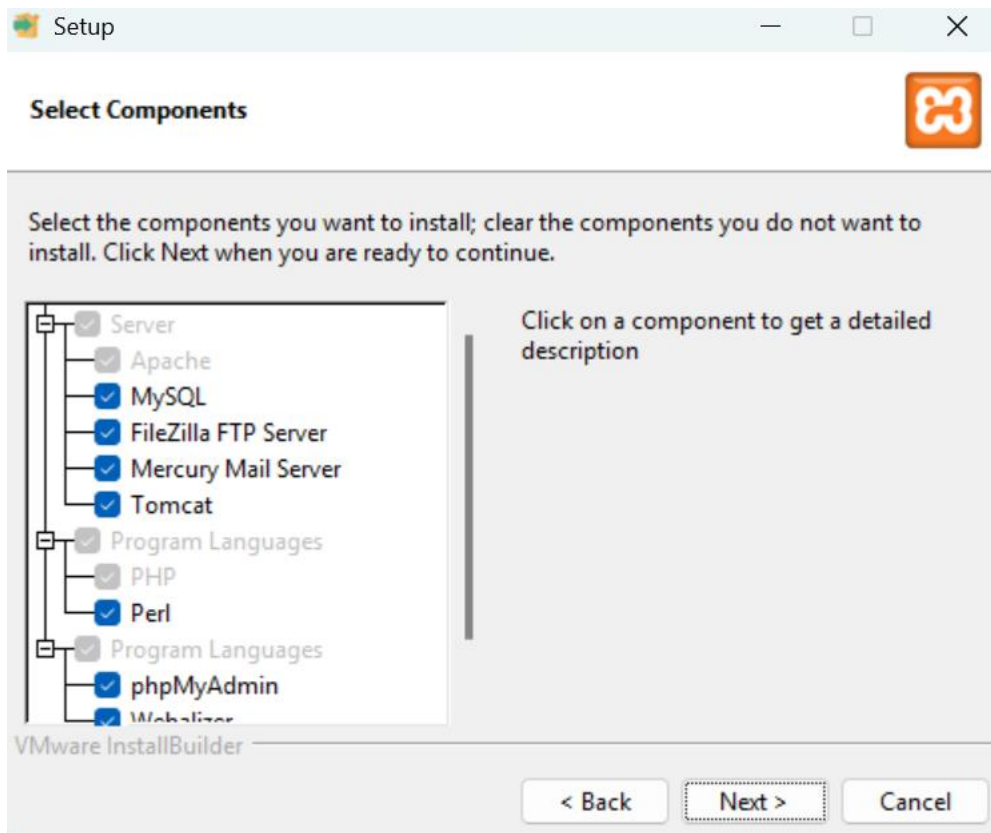
Visual Studio Code (VS Code) is a lightweight and powerful code editor developed by Microsoft.

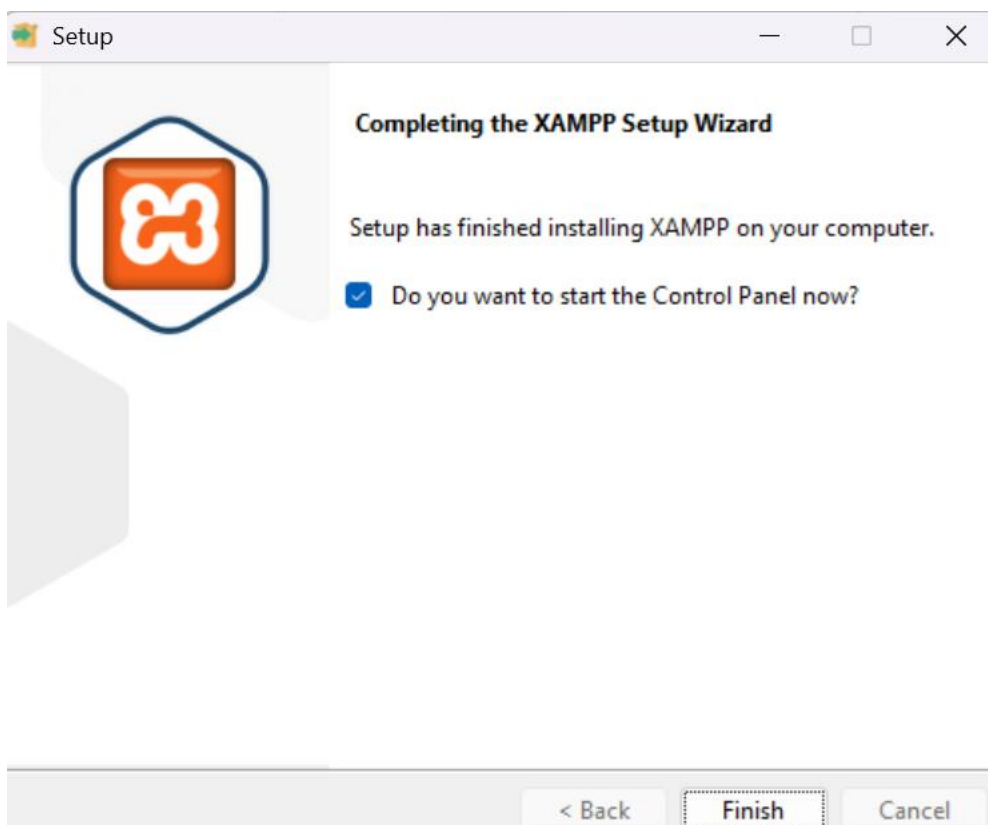
Uses

- Write PHP code
- Write HTML, CSS, and JavaScript
- Edit Laravel files
- Debug applications
- Install useful extensions

XAMPP Installation:







XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

XAMPP Control Panel v3.3.0

Modules

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	7088 9944	80, 443	Stop Admin Config Logs
☐	MySQL	2472	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

12:42:03 AM [main] All prerequisites found
12:42:03 AM [main] Initializing Modules
12:42:03 AM [main] Starting Check-Timer
12:42:03 AM [main] Control Panel Ready
12:51:39 AM [Apache] Attempting to start Apache app...
12:51:39 AM [Apache] Status change detected: running
12:51:41 AM [mysql] Attempting to start MySQL app...
12:51:43 AM [mysql] Status change detected: running

Config Netstat Shell Explorer Services Help Quit

New tab localhost / 127.0.0.1 | phpMyAdmin

localhost/phpmyadmin/

phpMyAdmin

Recent Favorites

- New
- information_schema
- mysql
- performance_schema
- phpmyadmin
- test

Server: 127.0.0.1

Databases SQL Status User accounts Export Import Settings More

General settings

Server connection collation: utf8mb4_unicode_ci

More settings

Appearance settings

Language: English

Theme: pmahomme View all

Database server

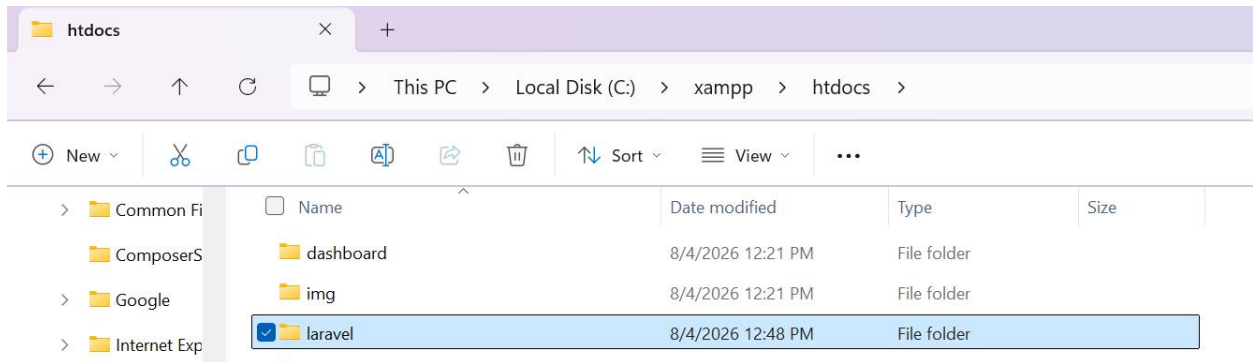
- Server: 127.0.0.1 via TCP/IP
- Server type: MariaDB
- Server connection: SSL is not being used
- Server version: 10.4.32-MariaDB - mariadb.org binary distribution
- Protocol version: 10
- User: root@localhost
- Server charset: UTF-8 Unicode (utf8mb4)

Web server

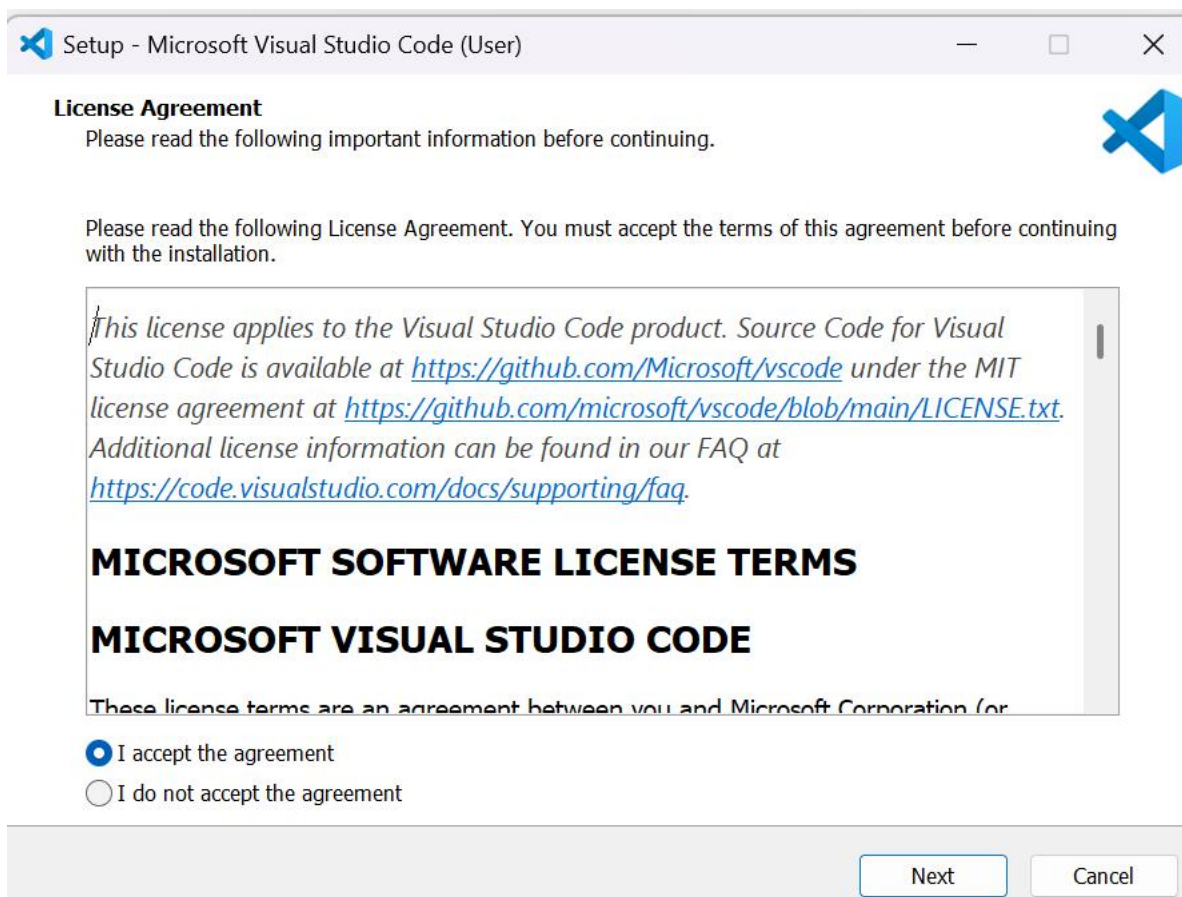
- Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30
- Database client version: libmysql - mysqlnd 8.0.30
- PHP extension: mysqli curl mbstring

Console

Htdocs Folder:



VS Code Installation:



Select Destination Location

Where should Visual Studio Code be installed?



Setup will install Visual Studio Code into the following folder.

To continue, click Next. If you would like to select a different folder, click Browse.

C:\Users\Pakistan_1st\AppData\Local\Programs\Microsoft VS Code

Browse...

At least 991.4 MB of free disk space is required.

Back

Next

Cancel

Ready to Install

Setup is now ready to begin installing Visual Studio Code on your computer.



Click Install to continue with the installation, or click Back if you want to review or change any settings.

Destination location:

C:\Users\Pakistan 1st\AppData\Local\Programs\Microsoft VS Code

Start Menu folder:

Visual Studio Code

Additional tasks:

Other:

- Register Code as an editor for supported file types
- Add to PATH (requires shell restart)

Back

Install

Cancel

Setup - Microsoft Visual Studio Code (User)

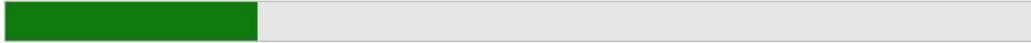
Installing

Please wait while Setup installs Visual Studio Code on your computer.



Extracting files...

C:\...\Local\Programs\Microsoft VS Code\e4c7e7b1d6\resources\app\extensions\copilot\dist\tree-sitter.wasm



Cancel

Completing the Visual Studio Code Setup Wizard

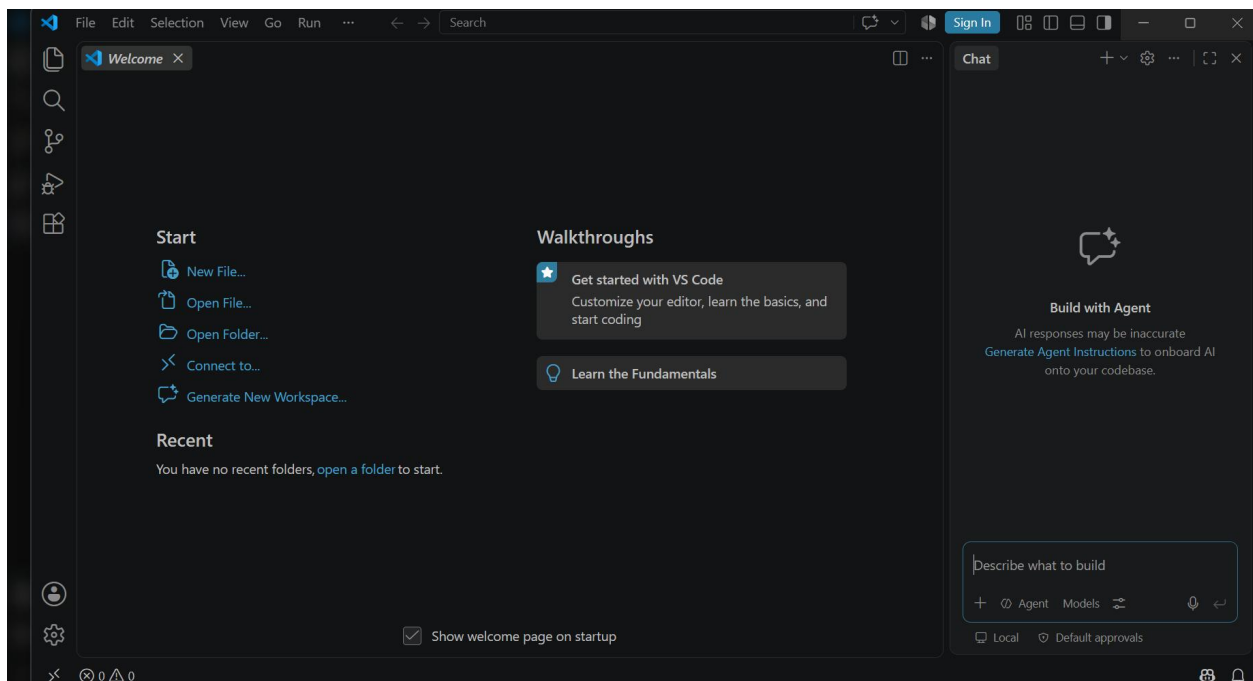
Setup has finished installing Visual Studio Code on your computer. The application may be launched by selecting the installed shortcuts.

Click Finish to exit Setup.



☒ Launch Visual Studio Code

Finish



Composer:



[Home](#) | [Getting Started](#) | [Download](#) | [Documentation](#) | [Browse Packages](#)

Download Composer Latest: v2.10.2



PRIVATE PACKAGIST

Secure & reliable package repository

Windows Installer

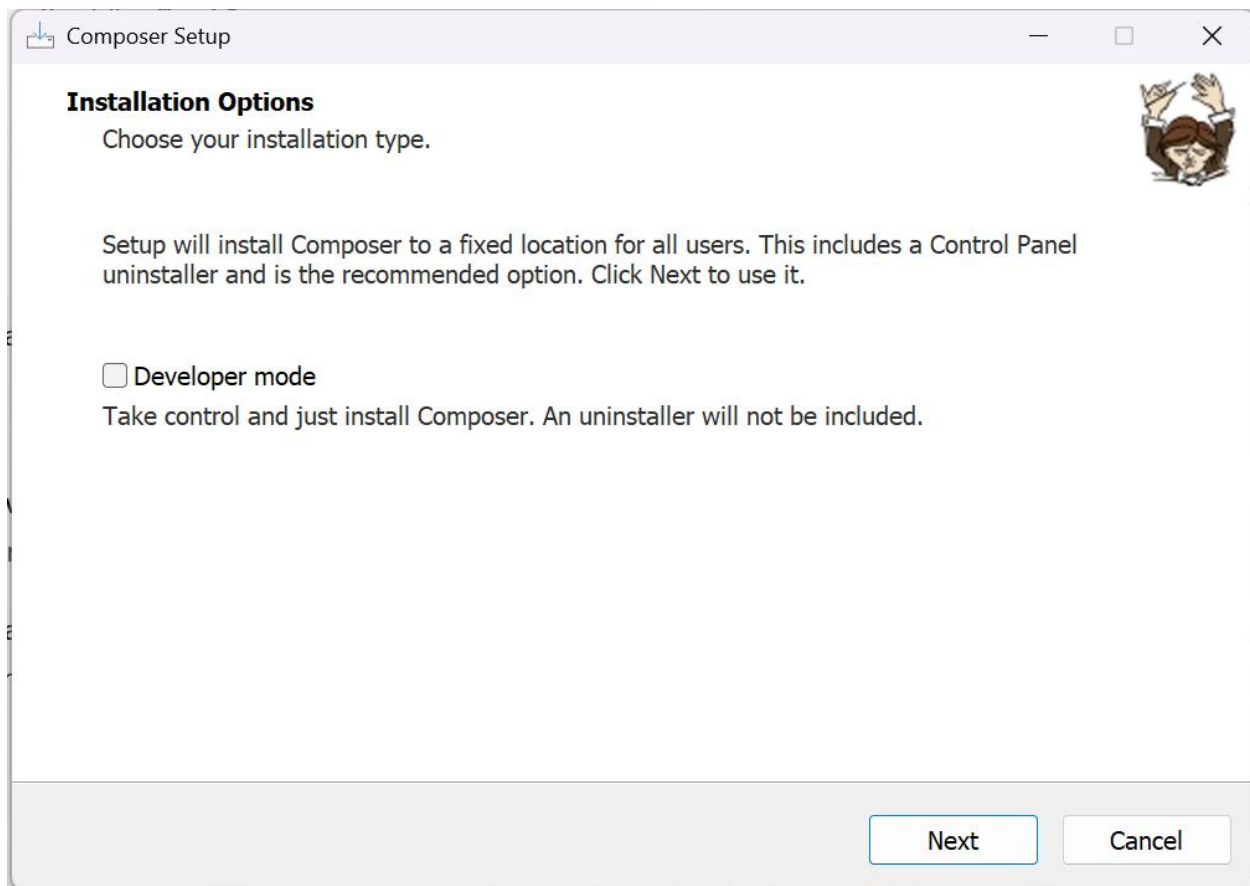
The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest composer version whenever it is executed.

Command-line installation

To quickly install Composer in the current directory, run the following script in your terminal. To automate the installation, use [the guide on installing Composer programmatically](#).

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') === 'c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eaa
php composer-setup.php
php -r "unlink('composer-setup.php');"
```





Composer Setup



Settings Check

We need to check your PHP and other settings.



Choose the command-line PHP you want to use:

C:\xampp\php\php.exe



Browse...



Add this PHP to your path?

Back

Next

Cancel

Ready to Install

Setup is now ready to download Composer and install it on your computer.



Please review these settings. Click Install to continue with the installation.

PHP version 8.0.30
C:\xampp\php\php.exe

Proxy: none

Add to System path:
C:\xampp\php

Back

Install

Cancel

Information

Please read the following information before continuing.



Important

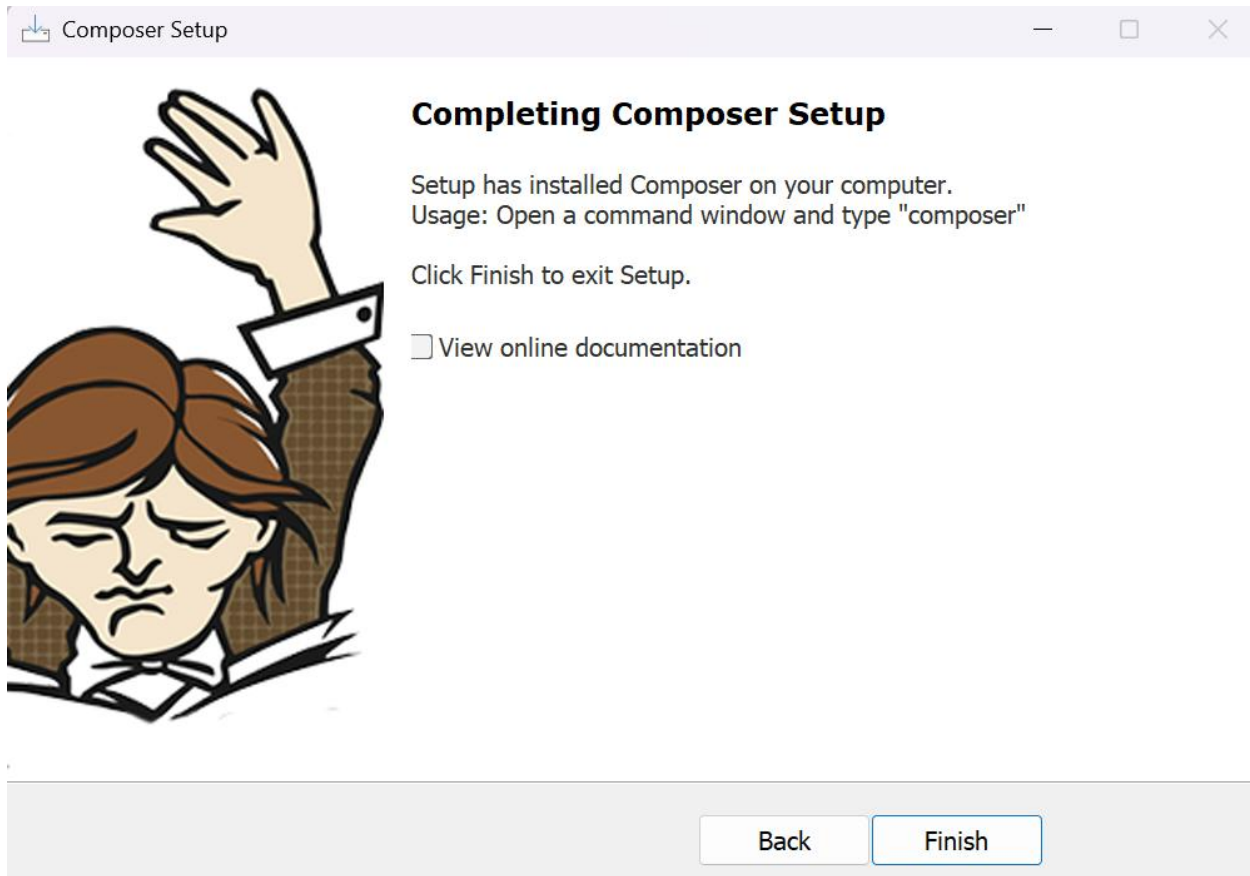
You must open a new command window to use Composer for the first time, because your environment has changed and running programs may not be aware of this.

If this does not work, you will have to do one of the following:

- Close all File Explorer windows, then open a new command window. OR
- Logoff and Logon again, then open a new command window.

As a last resort, you may need to restart your computer.

Next



```
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Pakistan 1st>composer

Composer version 2.10.2 2026-07-01 11:24:45

Usage:
  command [options] [arguments]

Options:
  -h, --help                Display help for the given command. When no command is given display help for the list command
  -q, --quiet               Do not output any message
  -V, --version             Display this application version
  --ansi|--no-ansi          Force (or disable --no-ansi) ANSI output
  -n, --no-interaction      Do not ask any interactive question
  --profile                 Display timing and memory usage information
  --no-plugins              Whether to disable plugins.
  --no-scripts              Skips the execution of all scripts defined in composer.json file.
  -d, --working-dir=WORKING-DIR If specified, use the given directory as working directory.
  --no-cache                Prevent use of the cache
  -v|vv|vvv, --verbose      Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and 3 for debug

Available commands:
  about      Shows a short information about Composer
  archive    Creates an archive of this composer package
  audit      Checks for security vulnerability advisories for installed packages
  browse     [home] Opens the package's repository URL or homepage in your browser
```

```
Command Prompt
C:\Users\Pakistan 1st>cd C:\xampp\htdocs
C:\xampp\htdocs>laravel new internship

Laravel

Creating a "laravel/laravel" project at "./internship"
Cannot use laravel/laravel's latest version v13.8.0 as it requires php ^8.3 which is not satisfied by your platform.
Installing laravel/laravel (v12.12.2)
Failed to download laravel/laravel from dist: The zip extension and unzip/7z commands are both missing, skipping.
The php.ini used by your command-line PHP is: C:\xampp\php\php.ini
Source fallback is disabled. Not trying alternative sources.

In ZipDownloader.php line 81:

The zip extension and unzip/7z commands are both missing, skipping.
The php.ini used by your command-line PHP is: C:\xampp\php\php.ini

create-project [-s|--stability STABILITY] [--prefer-source] [--prefer-dist] [--prefer-install PREFER-INSTALL] [--repository REPOSITORY]
[--repository-url REPOSITORY-URL] [--add-repository] [--require REQUIRE] [--dev] [--no-dev] [--no-custom-installers] [--no-scripts]
[--no-progress] [--no-secure-http] [--keep-vcs] [--remove-vcs] [--no-install] [--no-audit] [--audit-format AUDIT-FORMAT] [--no-security-blocking]
[--no-blocking] [--ignore-platform-req IGNORE-PLATFORM-REQ] [--ignore-platform-reqs] [--ask] [--] [<package> [<directory> [<version>]]]

C:\xampp\htdocs>
```

```
Command Prompt - php artit
> @php artisan key:generate --ansi
INFO Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi
INFO Preparing database.

Creating migration table ..... 25.92ms DONE
INFO Running migrations.

0001_01_01_000000_create_users_table ..... 95.07ms DONE
NE
0001_01_01_000001_create_cache_table ..... 60.59ms DONE
0001_01_01_000002_create_jobs_table ..... 80.86ms DONE
NE

INFO Application ready! Build something amazing.

C:\xampp\htdocs>cd C:\xampp\htdocs\internship
C:\xampp\htdocs\internship>php artisan serve
INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

2026-08-04 13:25:26 / ..... ~ 1s
2026-08-04 13:25:28 /favicon.ico ..... ~ 0.59ms
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

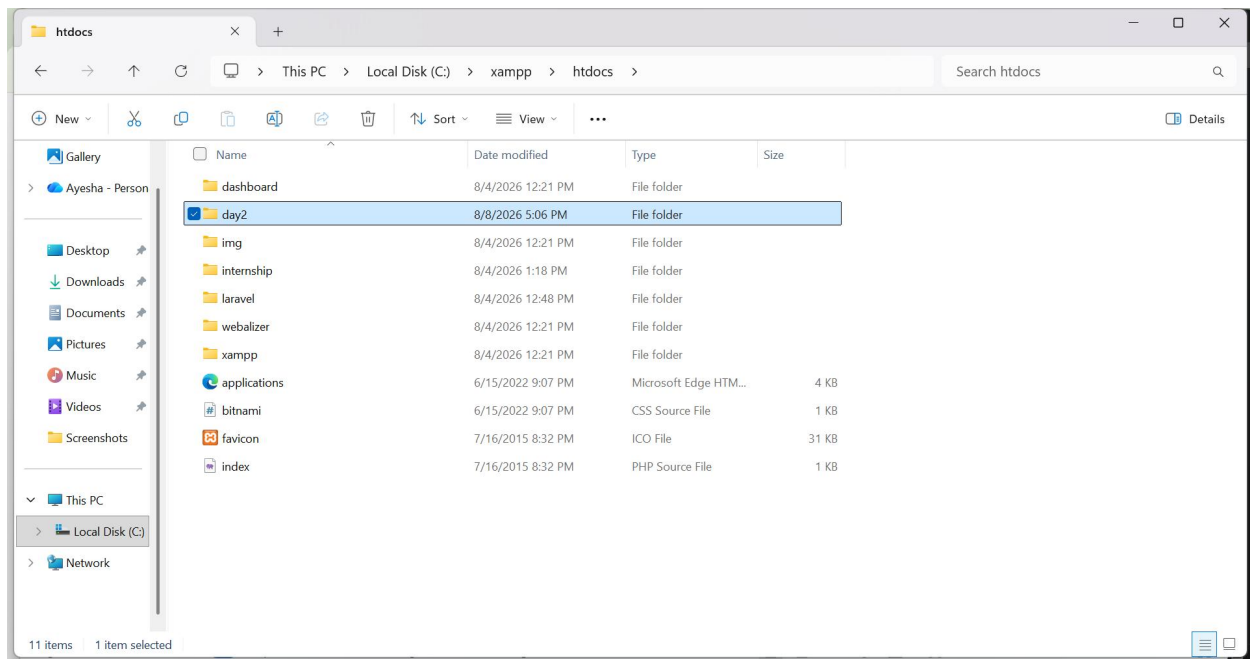
PS C:\Users\Pakistan 1st> php --version
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
PS C:\Users\Pakistan 1st> composer --version
Composer version 2.10.2 2026-07-01 11:24:45
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
PS C:\Users\Pakistan 1st> laravel --version
Laravel Installer 4.5.1
PS C:\Users\Pakistan 1st> |
```

DAY 2 — PHP Basics (Variables, Operators, Loops, Arrays, Functions)

1.PHP

PHP (Hypertext Preprocessor) is a server-side scripting language used to create dynamic websites and web applications.

Create PHP File:



2. Syntax

Syntax is the set of rules used to write PHP code correctly.

Example:

Code:

```
index.php
1  <?php
2  echo "Hello World";
3  ?>
```

Output:

Hello World

3. Variable

A **variable** is used to store data in a program. In PHP, a variable starts with \$.

Code:

```
Welcome index.php X
index.php > ...
1  <?php
2
3  $name = "Ayesha";
4  $age = 21;
5  $department = "Software Engineering";
6
7  echo "Name: " . $name . "<br>";
8  echo "Age: " . $age . "<br>";
9  echo "Department: " . $department;
10
11  ?>
```

Output:



4. Data Type

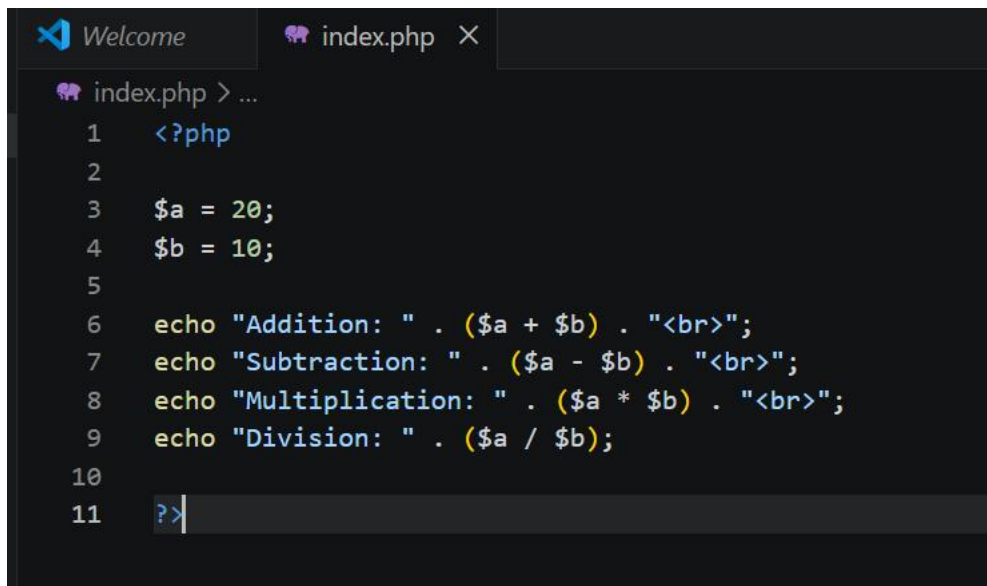
A **data type** defines what kind of data a variable stores.

Common PHP data types:

- String
- Integer
- Float
- Boolean
- Array
- Object
- NULL

5. Operator

An **operator** is a symbol used to perform an operation on values.

A screenshot of a code editor with a dark theme. The top bar shows two tabs: 'Welcome' and 'index.php'. The 'index.php' tab is active. The code is written in PHP and performs arithmetic operations on variables \$a and \$b. The code is as follows:

```
1 <?php
2
3 $a = 20;
4 $b = 10;
5
6 echo "Addition: " . ($a + $b) . "<br>";
7 echo "Subtraction: " . ($a - $b) . "<br>";
8 echo "Multiplication: " . ($a * $b) . "<br>";
9 echo "Division: " . ($a / $b);
10
11 ?>
```

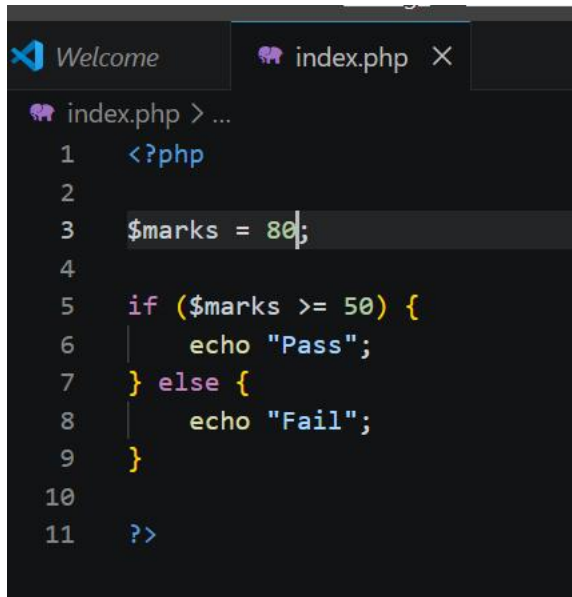
Output:

```
Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2
```

6. If-Else

If-else is a conditional statement used to execute different code depending on whether a condition is true or false.

Code:

A screenshot of a code editor window. The title bar shows 'Welcome' and 'index.php'. The code is as follows:

```
1 <?php
2
3 $marks = 80;
4
5 if ($marks >= 50) {
6     echo "Pass";
7 } else {
8     echo "Fail";
9 }
10
11 ?>
```

Output:

The output of the PHP code is 'Pass', displayed on a light green background.

7. Loop

A **loop** repeatedly executes a block of code until a specified condition is met.

Types:

- for
- while
- do-while
- foreach

Code:

```
Welcome index.php X
index.php > ...
1  <?php
2
3  for ($i = 1; $i <= 10; $i++) {
4      echo $i . "<br>";
5  }
6
7  ?>
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

8. Array

An **array** is a variable that can store multiple values.

Code:

```
index.php > ...
1  <?php
2
3  $students = ["Ayesha1", "Ayesha2", "Ayesha3", "Ayesha4"];
4
5  foreach ($students as $student) {
6      echo $student . "<br>";
7  }
8
9  ?>
```

Output:

```
Ayesha1  
Ayesha2  
Ayesha3  
Ayesha4
```

9. Function

A **function** is a reusable block of code that performs a specific task.

Code:

```
index.php > ...  
1  <?php  
2  
3  function add($a, $b)  
4  {  
5      return $a + $b;  
6  }  
7  
8  $result = add(30, 10);  
9  
10 echo "Result: " . $result;  
11  
12 ?>
```

Output:

```
Result: 40
```

10. Echo

echo is used to **display/output data** in PHP.

Code:

```
index.php
1  <?php
2  echo "Ayesha";
3  ?>
```

Output:

Ayesha

Day 3 — PHP OOP (Object-Oriented Programming)

1. OOP

Object-Oriented Programming (OOP) is a programming approach that organizes code using classes and objects.

2. Class

A class is a blueprint or template used to create objects.

Code:

```
class_object.php > ...
1  <?php
2
3  class Student
4  {
5      public $name;
6      public $age;
7      public $department;
8
9      public function displayInfo()
10     {
11         echo "Student Name: " . $this->name . "<br>";
12         echo "Age: " . $this->age . "<br>";
13         echo "Department: " . $this->department;
14     }
15 }
16
17 ?>
```

3. Object

An object is an instance of a class. It is created using the new keyword.

Code:


```
object.php > ...
3  class Student
4  {
5      public $name;
6      public $age;
7      public $department;
8
9      public function displayInfo()
10     {
11         echo "Student Name: " . $this->name . "<br>";
12         echo "Age: " . $this->age . "<br>";
13         echo "Department: " . $this->department;
14     }
15 }
16
17 $student = new Student();
18
19 $student->name = "Ayesha";
20 $student->age = 21;
21 $student->department = "Software Engineering";
22
23 $student->displayInfo();
24
25 ?>
```

Output:

```
Student Name: Ayesha
Age: 21
Department: Software Engineering
```

4. INHERITANCE

Inheritance allows a child class to use the properties and methods of a parent class.

PHP uses the extends keyword for inheritance.

Code:

```
inheritance.php > ...
1  <?php
2
3  // Parent Class
4  class Person
5  {
6      public $name;
7
8      public function introduce()
9      {
10         echo "My name is " . $this->name . "<br>";
11     }
12 }
13
14 // Child Class
15 class Student extends Person
16 {
17     public $department;
18
19     public function showDepartment()
20     {
21         echo "Department: " . $this->department;
22     }
23 }
24
25 // Create object of Student
26 $student = new Student();
27
28 $student->name = "Ayesha";
```

```
28 $student->name = "Ayesha";
29 $student->department = "Software Engineering";
30
31 // Methods from Parent and Child
32 $student->introduce();
33 $student->showDepartment();
34
35 ?>
```

Output:



My name is Ayesha
Department: Software Engineering

5. TRAITS

A trait is a mechanism in PHP used to reuse methods in multiple classes.

A trait is created using the trait keyword and included in a class using use.

Code:

```
trait.php > ...
1  <?php
2
3  trait Message
4  {
5      public function hello()
6      {
7          echo "Hello from the Trait!";
8      }
9  }
10
11 class Student
12 {
13     use Message;
14 }
15
16 $student = new Student();
17
18 $student->hello();
19
20 ?>
```

Output:

← → ↻ ⓘ localhost/day3/trait.php

Hello from the Trait!

6. TRAIT WITH MULTIPLE CLASSES

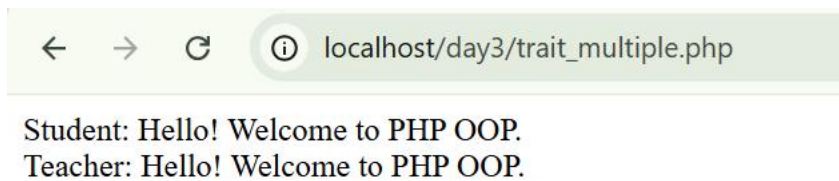
This shows why traits are useful.

Code:

```
trait_multiple.php > ...
1  <?php
2
3  trait Message
4  {
5      public function hello()
6      {
7          echo "Hello! Welcome to PHP OOP.<br>";
8      }
9  }
10
11 class Student
12 {
13     use Message;
14 }
15
16 class Teacher
17 {
18     use Message;
19 }
20
21 $student = new Student();
22 $teacher = new Teacher();
```

```
23
24     echo "Student: ";
25     $student->hello();
26
27     echo "Teacher: ";
28     $teacher->hello();
29
30     ?>
```

Output:



Student: Hello! Welcome to PHP OOP.
Teacher: Hello! Welcome to PHP OOP.

7. NAMESPACE

A namespace is used to organize PHP classes and prevent naming conflicts between classes with the same name.

Code:

```
SchoolStudent.php > ...
1  <?php
2
3  namespace School;
4
5  class Student
6  {
7      public function show()
8      {
9          echo "This is a School Student.";
10     }
11 }
12
13 ?>
```

```

namespace.php > ...
1  <?php
2
3  require_once "SchoolStudent.php";
4
5  $student = new \School\Student();
6
7  $student->show();
8
9  ?>

```

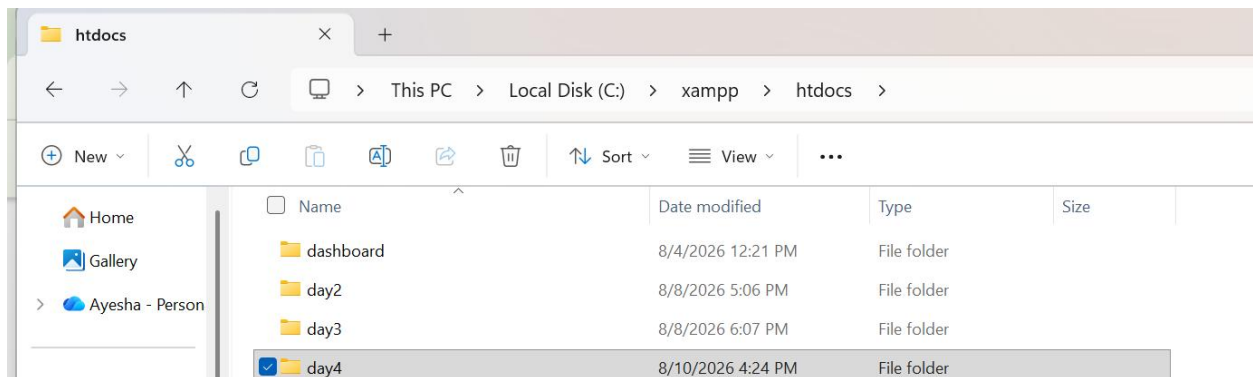
Output:



This is a School Student.

Day 4 —Laravel

1. Laravel Installation:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Pakistan 1st>php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies

C:\Users\Pakistan 1st>composer -V
Composer version 2.10.2 2026-07-01 11:24:45
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.

C:\Users\Pakistan 1st>|
```

```
Command Prompt

laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

81 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
> @php artisan key:generate --ansi

INFO Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi

INFO Preparing database.

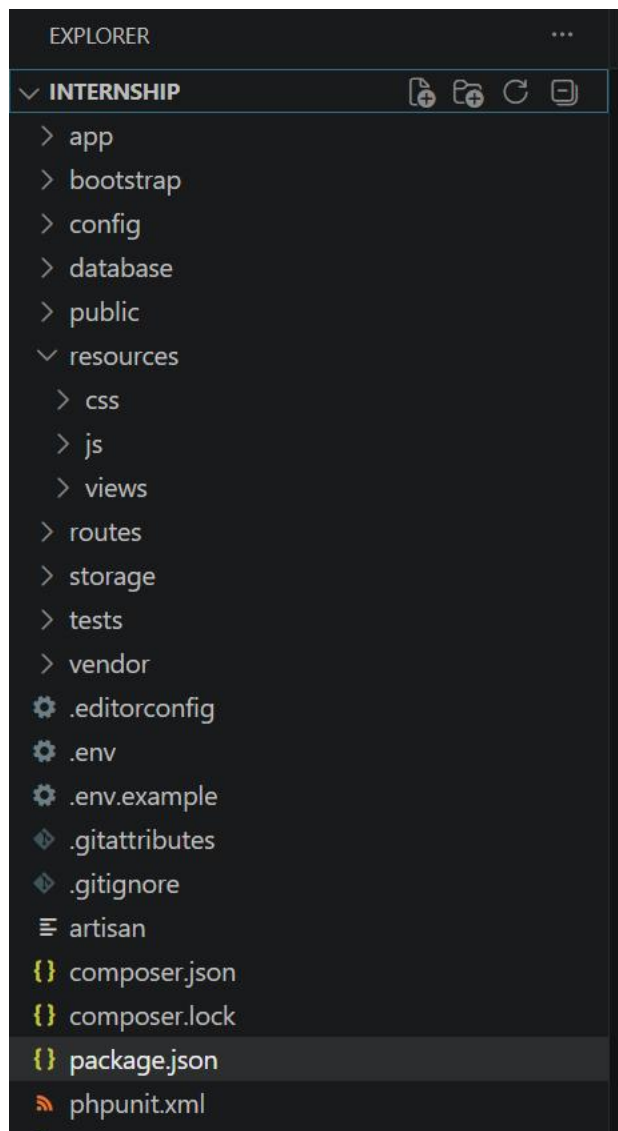
Creating migration table ..... 13.04ms DONE

INFO Running migrations.

0001_01_01_000000_create_users_table ..... 47.00ms DONE
0001_01_01_000001_create_cache_table ..... 29.29ms DONE
0001_01_01_000002_create_jobs_table ..... 44.06ms DONE

C:\Users\Pakistan 1st>|
```

2. Laravel Folder Structure:



3. Artisan Commands:

Artisan is Laravel's command-line interface used to perform development tasks.

```
C:\Users\Pakistan 1st>cd internship  
  
C:\Users\Pakistan 1st\internship>php artisan --version  
Laravel Framework 12.65.0
```


Command Prompt	
<code>queue:retry-batch</code>	Retry the failed jobs for a batch
<code>queue:work</code>	Start processing jobs on the queue as a daemon
<code>route</code>	
<code>route:cache</code>	Create a route cache file for faster route registration
<code>route:clear</code>	Remove the route cache file
<code>route:list</code>	List all registered routes
<code>sail</code>	
<code>sail:add</code>	Add a service to an existing Sail installation
<code>sail:install</code>	Install Laravel Sail's default Docker Compose file
<code>sail:publish</code>	Publish the Laravel Sail Docker files
<code>schedule</code>	
<code>schedule:clear-cache</code>	Delete the cached mutex files created by scheduler
<code>schedule:interrupt</code>	Interrupt the current schedule run
<code>schedule:list</code>	List all scheduled tasks
<code>schedule:run</code>	Run the scheduled commands
<code>schedule:test</code>	Run a scheduled command
<code>schedule:work</code>	Start the schedule worker
<code>schema</code>	
<code>schema:dump</code>	Dump the given database schema
<code>storage</code>	
<code>storage:link</code>	Create the symbolic links configured for the application
<code>storage:unlink</code>	Delete existing symbolic links configured for the application
<code>stub</code>	
<code>stub:publish</code>	Publish all stubs that are available for customization
<code>vendor</code>	
<code>vendor:publish</code>	Publish any publishable assets from vendor packages
<code>view</code>	
<code>view:cache</code>	Compile all of the application's Blade templates
<code>view:clear</code>	Clear all compiled view files

4. Environment Configuration:

Sort ▾ View ▾ ...				
☐	Name	Date modified	Type	Size
☐	resources	8/4/2026 1:17 PM	File folder	
☐	routes	8/4/2026 1:17 PM	File folder	
☐	storage	8/4/2026 1:17 PM	File folder	
☐	tests	8/4/2026 1:17 PM	File folder	
☐	vendor	8/4/2026 1:20 PM	File folder	
☐	.editorconfig	3/14/2026 9:54 AM	Editor Config Source ...	1 KB
☒	.env	8/4/2026 1:20 PM	ENV File	2 KB
☐	.env.example	8/4/2026 1:20 PM	EXAMPLE File	2 KB
☐	.gitattributes	3/14/2026 9:54 AM	Git Attributes Source ...	1 KB
☐	.gitignore	3/14/2026 9:54 AM	Git Ignore Source File	1 KB
☐	artisan	3/14/2026 9:54 AM	File	1 KB
☐	composer	3/14/2026 9:54 AM	JSON Source File	3 KB
☐	composer.lock	8/4/2026 1:18 PM	LOCK File	303 KB
☐	package	3/14/2026 9:54 AM	JSON Source File	1 KB
☐	phpunit	3/14/2026 9:54 AM	Microsoft Edge HTM...	2 KB
☐	README	3/14/2026 9:54 AM	Markdown Source File	4 KB
☐	vite.config.js	3/14/2026 9:54 AM	JSFile	1 KB

Generate Application Key:

```

C:\Users\Pakistan 1st\internship>php artisan key:generate

INFO Application key set successfully.
  
```

Run Laravel Project:

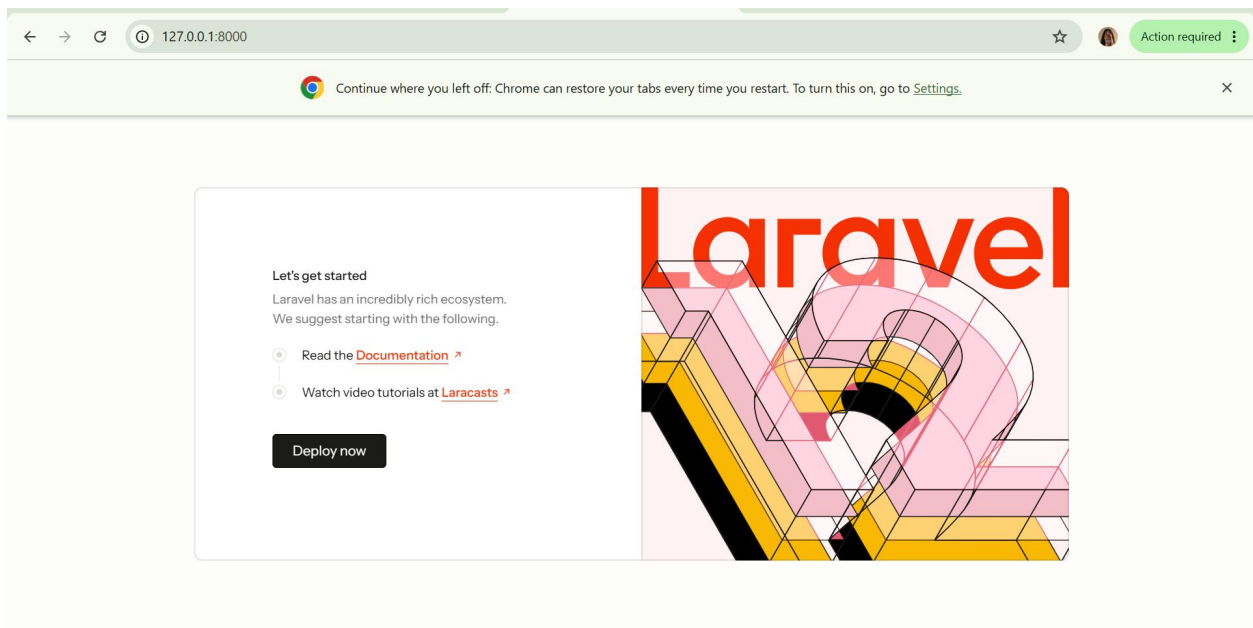
```

C:\Users\Pakistan 1st\internship>php artisan serve

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

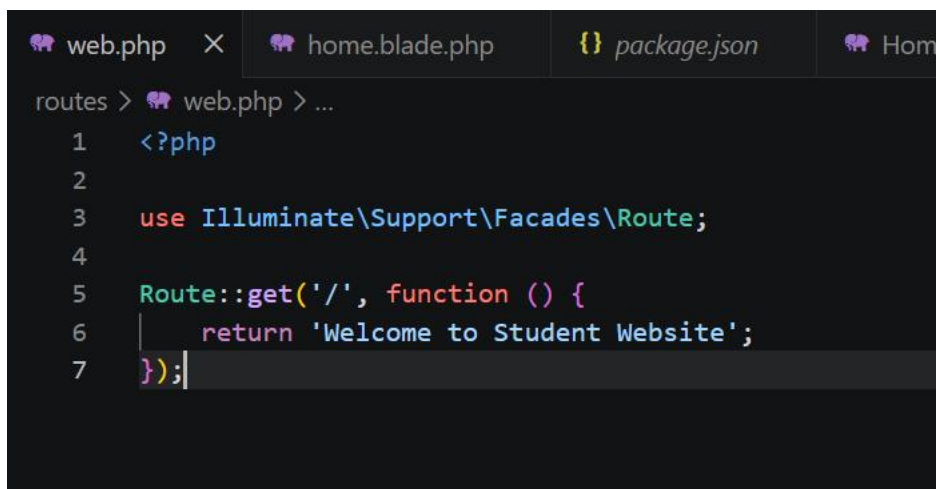
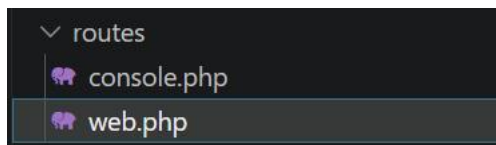
2026-08-10 16:49:32 / ..... ~ 502.06ms
2026-08-10 16:49:33 /favicon.ico ..... ~ 505.02ms
  
```



DAY 5 — Routing, Controllers, Blade Templates & Passing Data

1. ROUTING

Routing defines how a Laravel application responds to a particular URL.





Welcome to Student Website

MULTIPLE ROUTES:

```
routes > web.php > ...
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4
5  Route::get('/', function () {
6      return 'Home Page';
7  });
8
9  Route::get('/about', function () {
10     return 'About Page';
11 });
12
13 Route::get('/students', function () {
14     return 'Students Page';
15 });
16
17 Route::get('/contact', function () {
18     return 'Contact Page';
19 });
20
```



Home Page



About Page



Students Page



Contact Page

2.CONTROLLERS

A **Controller** is a PHP class used to organize the logic that handles requests from routes.

Instead of putting all logic directly inside web.php, we can use controllers.

Create Controller:

```
C:\Users\Pakistan 1st\internship>php artisan make:controller PageController  
INFO Controller [C:\Users\Pakistan 1st\internship\app\Http\Controllers\PageController.php] created successfully.
```

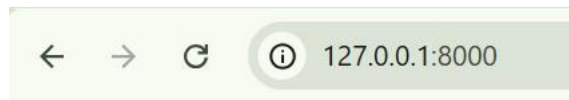
ADD METHODS TO CONTROLLER:

```
File Edit Selection View Go Run ... < -> internship
EXPLORER
INTERNSHIP
  app
    Http\Controllers
      Controller.php
      PageController.php
    Models
    Providers
    bootstrap
    config
    database
    public
    resources
      css
      js
      views
    routes
      console.php
      web.php
    storage
    tests
    vendor
    .editorconfig
    .env
    .env.example

app > Http > Controllers > PageController.php > PageController
1 <?php
2
3 namespace App\Http\Controllers;
4
5 class PageController extends Controller
6 {
7     public function home()
8     {
9         return 'Home Page';
10    }
11
12    public function about()
13    {
14        return 'About Page';
15    }
16
17    public function students()
18    {
19        return 'Students Page';
20    }
21
22    public function contact()
23    {
24        return 'Contact Page';
25    }
26 }
```

CONNECT ROUTES WITH CONTROLLER:

```
web.php x home.blade.php package.json PageController.php
routes > web.php
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use App\Http\Controllers\PageController;
5
6 Route::get('/', [PageController::class, 'home']);
7
8 Route::get('/about', [PageController::class, 'about']);
9
10 Route::get('/students', [PageController::class, 'students']);
11
12 Route::get('/contact', [PageController::class, 'contact']);
```



Home Page



About Page



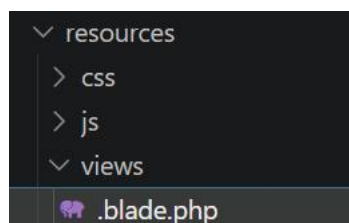
Students Page



Contact Page

3. BLADE TEMPLATES

Blade is Laravel's templating engine used to create dynamic HTML pages.



CREATE HOME PAGE:

```
resources > views > 🐘 home.blade.php > 📄 html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Student Website</title>
5  </head>
6  <body>
7  |
8  |   <h1>Welcome to Student Website</h1>
9  |
10 |   <p>This is the Home Page.</p>
11 |
12 </body>
13 </html>
```

CREATE OTHER PAGES:

```
resources > views > 🐘 about.blade.php > 📄 html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>About</title>
5  </head>
6  <body>
7  |
8  |   <h1>About Us</h1>
9  |
10 |   <p>This is the About Page.</p>
11 |
12 </body>
13 </html>
```



```
resources > views > 🐘 student.blade.php > 📄 html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Students</title>
5  </head>
6  <body>
7  |
8  |   <h1>Students</h1>
9  |
10 |   <p>Welcome to the Students Page.</p>
11 |
12 </body>
13 </html>
```

```
resources > views > 🐘 contactt.blade.php > 📄 html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Contact</title>
5  </head>
6  <body>
7  |
8  |   <h1>Contact Us</h1>
9  |
10 |   <p>Email: student@example.com</p>
11 |
12 </body>
13 </html>
```

RETURN BLADE VIEWS FROM CONTROLLER:

app > Http > Controllers > PageController.php > PageController

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  class PageController extends Controller
6  {
7      public function home()
8      {
9          return view('home');
10     }
11
12     public function about()
13     {
14         return view('about');
15     }
16
17     public function students()
18     {
19         return view('students');
20     }
21
22     public function contact()
23     {
24         return view('contact');
25     }
26 }
```

← → ↻ ⓘ 127.0.0.1:8000

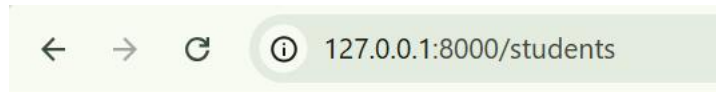
Welcome to Student Website

This is the Home Page.



About Us

This is the About Page.



Students

Welcome to the Students Page.



Contact Us

Email: student@example.com

4.PASSING DATA

Passing data means sending information from a controller to a Blade view so that the view can display dynamic information.

Modify Controller:

app > Http > Controllers > 🐞 PageController.php > 🛠 PageController

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  class PageController extends Controller
6  {
7      public function students()
8      {
9          $name = "Ayesha";
10         $department = "Software Engineering";
11
12         return view('students', compact('name', 'department'));
13     }
14 }
```

resources > views > 🐞 students.blade.php > 📄 html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Students</title>
5  </head>
6  <body>
7
8      <h1>Student Information</h1>
9
10     <p>Name: {{ $name }}</p>
11
12     <p>Department: {{ $department }}</p>
13
14 </body>
15 </html>
```

← → ↻ ⓘ 127.0.0.1:8000/students

Student Information

Name: Ayesha

Department: Software Engineering

